



Pashov Audit Group

Regnum Aurum PSM Vault Security Review

March 26th 2026 - March 30th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Regnum Aurum PSM Vault	4
5. Executive Summary	4
6. Findings	5
Medium findings	6
[M-01] Depositor can exploit withdrawal fees by draining savings reserves through swap	6
[M-02] Depositor receives yield from their own deposit fee	7
[M-03] Swap enables withdrawal fee manipulation sandwich on <code>redeem</code> / <code>withdraw</code>	9
Low findings	12
[L-01] <code>RateProvider</code> assumes savings token and underlying token have same decimal	12
[L-02] <code>RateProvider</code> incompatible with documented proxy deployment pattern	13
[L-03] <code>MaxDeposit</code> and <code>maxRedeem</code> do not reflect paused state	13
[L-04] Incorrect asset preview in <code>deepRebalance</code> may prevent expected rebalancing	15
[L-05] Setting <code>withdrawTimeLimit=0</code> does not guarantee expiring all pending request	18
[L-06] Rounding to zero in share conversion causes silent loss of user withdrawal value	19



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Regnum Aurum PSM Vault

Regnum Aurum PSM Vault is a Peg Stability Module that allows users to deposit assets to mint pmUSD stablecoins and redeem them back, featuring a withdrawal fee that scales with the pmUSD-to-total-assets ratio and a built-in swap between pmUSD and savings tokens. The RateProvider contract supplies the current net asset value for pmUSD pricing using on-chain signatures.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/psm-vault** repository was done by Pashov Audit Group, during which **BengalCatBalu, Tejas Warambhe, Said, lanrebayode77** engaged to review **Regnum Aurum PSM Vault**. A total of **9** issues were uncovered.

Protocol Summary

Project Name	Regnum Aurum PSM Vault
Protocol Type	Peg Stability Module
Timeline	March 26th 2026 - March 30th 2026

Review commit hash:

- [819f795e2af19adc1cf8ff1e9bb44f8833dda511](#)
(RegnumAurumAcquisitionCorp/psm-vault)

Fixes review commit hash:

- [5bd42b1af9f774ba03547b44d1861900f71458c0](#)
(RegnumAurumAcquisitionCorp/psm-vault)

Scope

`PSMVault.sol``RateProvider.sol`



6. Findings

Findings count

Severity	Amount
Medium	3
Low	6
Total findings	9

Summary of findings

ID	Title	Severity	Status
[M-01]	Depositor can exploit withdrawal fees by draining savings reserves through swap	Medium	Resolved
[M-02]	Depositor receives yield from their own deposit fee	Medium	Resolved
[M-03]	Swap enables withdrawal fee manipulation sandwich on <code>redeem</code> / <code>withdraw</code>	Medium	Resolved
[L-01]	<code>RateProvider</code> assumes savings token and underlying token have same decimal	Low	Resolved
[L-02]	<code>RateProvider</code> incompatible with documented proxy deployment pattern	Low	Resolved
[L-03]	MaxDeposit and maxRedeem do not reflect paused state	Low	Resolved
[L-04]	Incorrect asset preview in deepRebalance may prevent expected rebalancing	Low	Resolved
[L-05]	Setting <code>withdrawTimeLimit=0</code> does not guarantee expiring all pending request	Low	Resolved
[L-06]	Rounding to zero in share conversion causes silent loss of user withdrawal value	Low	Resolved



Medium findings

[M-01] Depositor can exploit withdrawal fees by draining savings reserves through swap

Severity

Impact: Medium

Likelihood: Medium

Description

The withdrawal fee scales linearly with the pmUSD-to-total-assets ratio:

```
// PSMVault.sol:846-857
uint256 pmUSDValue = baseBalance > pending ? baseBalance - pending : 0;
// + module value
uint256 range = maxWithdrawFee - minWithdrawFee;
feeBps = minWithdrawFee + Math.mulDiv(pmUSDValue, range, _totalAssets, Math.Rounding.Ceil);
```

The only implemented swap direction is `pmUSD → savingsToken`. Each swap increases the vault's pmUSD balance and reduces its savings balance, increasing `pmUSDValue / totalAssets` and pushing `feeBps` toward `maxWithdrawFee`.

After computing `amountOut`, `_swapBaseToSavings` calls `_checkReserveRatio`, which reverts if the post-swap savings ratio would fall below `reserveRatioFloor`:

```
// PSMVault.sol:534-536
_checkReserveRatio(amountOut);
```

This prevents a single swap from draining savings to zero. With `reserveRatioFloor = 0.20e18` (20%), the attacker drives the vault to 80% pmUSD, yielding:

```
feeBps = minWithdrawFee + 0.80 × (maxWithdrawFee - minWithdrawFee)
       = 300 + 0.80 × 1700 = 1660 bps (16.6%)
```

When `swapFee = 0`, each swap is a zero-cost composition change: the attacker sends pmUSD to the vault and receives savings tokens of equal value. Nothing leaves the attacker's net worth; only the vault's internal ratio shifts. The economic gain to the attacker comes entirely from the elevated withdrawal fees paid by other users.

Concrete example (10,000,000 TVL, attacker holds 5%, `reserveRatioFloor = 0.20e18`, `swapFee = 0`):



The table presents a comparison of the withdrawal fee structure and the potential financial impact of an attack on the savings reserves through the exploitation of withdrawal fees. It outlines two scenarios: a normal state with a savings ratio of 80% and a compromised state following an attack that reduces the savings ratio to 20%.

In the normal scenario, the `feeBps` is calculated as 3% plus 0.20 multiplied by 17%, resulting in a total fee of 6.4%. Conversely, after the attack, the `feeBps` increases significantly to 16.6%, calculated as 3% plus 0.80 multiplied by 17%. This substantial increase in the fee percentage reflects the vulnerability in the system that the attacker exploits.

The fee on a withdrawal of 1,000,000 is also affected by this change. Under normal conditions, the fee amounts to 64,000, while after the attack, it escalates to 166,000. This increase in fees directly impacts the financial outcome for the attacker.

Furthermore, the attacker's yield, which is calculated as a 5% share of the fees collected, shows a marked difference between the two scenarios. In the normal state, the yield is 3,200, but it rises to 8,300 after the attack, indicating a significant profit from the exploitation.

Notably, the attack cost is marked as zero in the compromised scenario, as the `swapFee` is set to 0. This detail highlights the ease with which an attacker can exploit the system without incurring any costs, further emphasizing the urgency of addressing this vulnerability.

Recommendation

Possible solution is to charge a dynamically higher swap fee when a swap worsens the savings ratio beyond a threshold, making large composition-distorting swaps costly rather than free.

[M-02] Depositor receives yield from their own deposit fee

Severity

Impact: Medium

Likelihood: Medium

Description

Shares are calculated on `pmUSDValueAfterFee`, but the full savings token amount — including the fee portion — is transferred into the vault before shares are minted:

```
// PSMVault.sol:452-467
if (depositFee > 0) {
    uint256 fee = Math.mulDiv(pmUSDValue, depositFee, MAX_BPS, Math.Rounding.Ceil);
    pmUSDValueAfterFee = pmUSDValue - fee;
    // Fee portion remains in vault as savings token balance, boosting totalAssets()
}

// Compute shares BEFORE transfer so totalAssets() reflects pre-deposit state
sharesOut = convertToShares(pmUSDValueAfterFee);
```



```
savingsToken.safeTransferFrom(msg.sender, address(this), savingsAmount); // full amount, incl. fee
_mint(receiver, sharesOut);
```

After `safeTransferFrom` executes, `totalAssets()` includes the fee portion. The depositor's freshly minted shares are now claims on a pool that already contains their own fee. Because they own `sharesOut / (totalSupply + sharesOut)` of the vault, they immediately recapture that fraction of the fee as yield on their position.

In an empty vault this is total: a depositor with `D = 100,000` and `depositFee = 5%` mints 95,000 shares on a vault that now holds 100,000 assets. They own 100% of `totalSupply`, so `convertToAssets(95,000) = 100,000` — the full deposit including the fee they ostensibly paid. Their effective deposit fee is zero.

The effective fee actually extracted from any depositor is:

```
effective_fee = D × df × T / (T + D × (1 - df))
```

where `T` is `totalAssets` before the deposit. For the first depositor, `T = 0` and the effective fee is always `0`. As `D` grows relative to `T`, the effective rate collapses regardless of the nominal `depositFee`.

The spec states that deposit fees generate protocol revenue for existing shareholders. In practice, a significant share of every deposit fee flows back to the depositor themselves through the instant appreciation of their newly minted shares.

Recommendation

Compute shares on the full `pmUSDValue` first, then take the fee in shares rather than in the asset value:

```
sharesOut = convertToShares(pmUSDValue); // full value
if (depositFee > 0) {
    uint256 feeShares = Math.mulDiv(sharesOut, depositFee, MAX_BPS, Math.Rounding.Ceil);
    sharesOut -= feeShares;
    // burn feeShares or transfer to a protocol treasury address
}
savingsToken.safeTransferFrom(msg.sender, address(this), savingsAmount);
_mint(receiver, sharesOut);
```

This way, the fee portion increases the share price for all existing holders without the depositor holding any claim to it.



[M-03] Swap enables withdrawal fee manipulation sandwich on `redeem` / `withdraw`

Severity

Impact: High

Likelihood: Low

Description

The dynamic withdrawal fee in `_computeWithdrawFee()` is computed from the vault's token composition, `pmUSDValue / totalAssets`.

```
function _computeWithdrawFee() internal view returns (uint256 feeBps) {
    uint256 _totalAssets = totalAssets();
    if (_totalAssets == 0) return minWithdrawFee;

    uint256 pending = _pendingRewardsVirtual();
    uint256 baseBalance = baseAsset.balanceOf(address(this));
    uint256 pmUSDValue = baseBalance > pending ? baseBalance - pending : 0;

    if (address(baseAssetModule) != address(0)) {
        uint256 moduleShares = IERC20(address(baseAssetModule)).balanceOf(address(this));
        if (moduleShares > 0) {
            pmUSDValue += baseAssetModule.convertToAssets(moduleShares);
        }
    }

    uint256 range = maxWithdrawFee - minWithdrawFee;
    >>> feeBps = minWithdrawFee + Math.mulDiv(pmUSDValue, range, _totalAssets,
    Math.Rounding.Ceil);

    // Clamp to maxWithdrawFee as safety
    if (feeBps > maxWithdrawFee) feeBps = maxWithdrawFee;
}
```

The one-way `swap` from `pmUSD` to `savingsToken` is permissionless and shifts vault composition by increasing `pmUSD` balance and decreasing `savingsToken` balance. Neither `redeem()` nor `withdraw()` has any slippage protection parameters, so the user cannot protect against fee manipulation. Under certain conditions, the attack can become profitable or at least the cost of manipulation (maximum swap fee is 1%) is cheap compared to victim loss (maximum withdraw fee is 20%).



Consider this scenario:

1. Initial state:
2. Vault holds 500,000e18 `pmUSD`, 500,000e18 `savingsToken` (rate = 1e18 for simplicity)
3. `totalAssets` = 1,000,000e18
4. `minWithdrawFee` = 50, `maxWithdrawFee` = 1000, `swapFee` = 15 (taken from test default values).
5. `reserveRatioFloor` = 0.20e18 (20%)
6. Victim locked 100,000 shares ($\approx 100,000\text{e18}$ pmUSD value) via `requestExit()`
7. Attacker front-runs with `swap` 300,000e18 `pmUSD` to `savingsToken` :
8. 300,000e18 `pmUSD` transferred to vault $\rightarrow$ `baseBalance` = 800,000e18
9. `fee` = $\text{ceil}(300,000\text{e18} \times 15 / 10,000) = 450\text{e18}$
10. `amountInAfterFee` = 299,550e18
11. `amountOut` = $\text{floor}(299,550\text{e18} \times 1\text{e18} / 1\text{e18}) = 299,550\text{e18}$ `savingsToken`
12. `_checkReserveRatio(299,550e18)` passed :
13. `remainingSavings` = $500,000\text{e18} - 299,550\text{e18} = 200,450\text{e18}$
14. ratio = $200,450\text{e18} / 1,000,450\text{e18} = 20.04\% \geq 20\% \checkmark$ (passes)
15. Assumption: `_rebalance()`, with rebalancer = address(0), or for any reason (such as delay), the rebalance is skipped.

Post-swap vault: 800,000e18 pmUSD, 200,450e18 savingsToken. totalAssets $\approx 1,000,450\text{e18}$.

1. Victim's `redeem` 100,000 shares execute:
2. `_computeWithdrawFee()` :
3. `pmUSDValue` = 800,000e18
4. `totalAssets` = 1,000,450e18
5. `feeBps` = $50 + \text{ceil}(800,000\text{e18} \times 950 / 1,000,450\text{e18}) = 50 + 760 = 810$ bps (8.10%)
6. `feeAmount` = $\text{ceil}(100,000\text{e18} \times 810 / 10,000) = 8,100\text{e18}$
7. `entitlementAfterFee` = 91,900e18
8. If not manipulated, normal fee computation:
9. `pmUSDValue` = 500,000e18
10. `feeBps` = $50 + \text{ceil}(500,000\text{e18} \times 950 / 1,000,000\text{e18}) = 50 + 475 = 525$ bps (5.25%)
11. Victim's expected fee: $\text{ceil}(100,000\text{e18} \times 525 / 10,000) = 5,250\text{e18}$

Victim receives 91,900e18 instead of 94,750e18. Victim loses 2,850e18 pmUSD from fee manipulation.



Victim loss / attacker cost = $2,850 / 450 = 6.3\times$.

Besides active fee manipulation, the following can also change:

- Oracle rate: Savings token valuation changes via Chainlink and `convertToAssets` fluctuations.
- Withdrawal fee: Governance can change `minWithdrawFee` / `maxWithdrawFee`.
- Rate provider: Governance can swap the entire Oracle contract.

`_distributeProportional()` is also an affected function: it determines how the net pmUSD entitlement is split across `savingsToken`, `baseAssetModule` shares, and raw pmUSD in proportion to the vault's live composition at claim time. A swap between `requestExit()` and claim drains the savings token reserve, causing the user to receive fewer savings tokens and more pmUSD (or module shares) than expected at request time, regardless of the total value.

Recommendations

Consider adding slippage protection on `redeem` / `withdraw`. Consider adding both `minPmUSDOut` (aggregate pmUSD floor) and optional per-token minimum parameters (e.g., `minSavingsTokenOut`, `minBaseAssetOut`, `minModuleTokenOut`) to `redeem()` and `withdraw()`. Callers that do not require per-token guarantees pass 0. This protects against both total-value slippage and unwanted token composition outcomes.



Low findings

[L-01] `RateProvider` assumes savings token and underlying token have same decimal

Description

`_computeRate` computes the pmUSD value of one savings token using the formula:

```
// RateProvider.sol:154-165
uint256 sharePrice = ISavingsToken(savingsToken).convertToAssets(10 ** savingsTokenDecimals);
uint256 underlyingUSD = _getChainlinkPrice(); // USD per 1 underlying token, 18 dec

rate = Math.mulDiv(sharePrice, underlyingUSD, 10 ** savingsTokenDecimals, rounding);
```

`convertToAssets(10 ** savingsTokenDecimals)` returns underlying tokens denominated in the **underlying's** decimals (`U`). The Chainlink price `underlyingUSD` represents the value of one **full** underlying token — i.e., 10^U raw units — scaled to 18 decimals. The correct normalization therefore requires dividing by 10^U :

```
rate = sharePrice [U dec] * underlyingUSD [18 dec] / 10^U
```

The code divides by `10 ** savingsTokenDecimals` (10^S) instead. The two are equal only when $S = U$. If they differ, the rate carries a permanent factor of $10^{(U - S)}$:

The table provides an analysis of the decimal assumptions for the `savingsToken` and the underlying token in the context of the `_computeRate` function. It highlights the potential errors that can arise when these decimal values do not align.

In the first scenario, both the `savingsToken` and the underlying token have a decimal value of 18, specifically for `scrVUSD` and `crvUSD`. This alignment results in an error factor of 1x, indicating that the rate calculation is correct.

The second scenario presents a mismatch where the `savingsToken` has a decimal value of 18, while the underlying token, `USDC`, has a decimal value of 6. This discrepancy leads to an error factor of 10^{-12} , which implies that the computed rate is approximately 0. Consequently, the function `ZeroRate` will always revert due to this condition.

In the final scenario, the `savingsToken` has a decimal value of 6, while the underlying token has a decimal value of 18. This situation results in an error factor of 10^{12} , indicating that the computed rate is wildly inflated due to the decimal mismatch.

Overall, the table emphasizes the critical importance of ensuring that the decimal values of the `savingsToken` and the underlying token are consistent to avoid significant errors in rate calculations.



From the NatSpec comment at line 22, it states "Handles tokens with different decimals (e.g., 18 for scrvUSD, 6 for savings USDC vaults)" and the `README`, we can assume the contract is intended to support any ERC-4626 savings vault. The ERC-4626 standard does not require share decimals to match underlying decimals.

The contract's own NatSpec states it "Handles tokens with different decimals (e.g., 18 for scrvUSD, 6 for savings USDC vaults)," indicating the intent to support varied decimal configurations. This makes the assumption that share decimals equal underlying decimals a documented but violated contract invariant. A common real-world trigger is OpenZeppelin's `_decimalsOffset()` pattern, where an ERC-4626 vault sets `vault.decimals() = underlying.decimals() + offset` (e.g., 12-decimal offset over 6-decimal USDC) to mitigate share inflation attacks. Any such vault integrated as the savings token would produce a rate error of `10offset`.

Recommendation

Cache `underlyingDecimals` from `ISavingsToken(_savingsToken).asset()` in the constructor and use it as the divisor.

[L-02] `RateProvider` incompatible with documented proxy deployment pattern

Description

The `RateProvider` contract contains a `@dev` NatSpec comment stating:

```
// ...  
/// @dev Deployed behind TransparentUpgradeableProxy for upgradeability without touching  
PSMVault.  
contract RateProvider is IRateProvider, Ownable {  
// ...
```

However, the contract is architecturally incompatible with proxy deployment; there is no `initialize()` function and `Ownable(msg.sender)` in the constructor.

If proxy upgradeability is desired, refactor the contract to add an `initialize()` function and replace all `immutable` variables with mutable storage variables set in the initializer.

[L-03] `MaxDeposit` and `maxRedeem` do not reflect paused state

Description

The `maxDeposit` and `maxRedeem` functions do not account for the protocol's paused state, leading to inconsistencies between view functions and actual execution behavior.

Currently, both functions only enforce internal constraints such as caps and timing conditions:



```
function maxDeposit(address) external view returns (uint256) {
    uint256 currentBalance = savingsToken.balanceOf(address(this));
    if (currentBalance >= depositCap) return 0;
    return depositCap - currentBalance;
}
```

```
function maxRedeem(address owner) public view returns (uint256) {
    RedeemRequest storage req = redeemRequests[owner];
    uint256 exitTime = req.exitTime;
    if (exitTime == 0 || block.timestamp < exitTime) return 0;
    if (block.timestamp > exitTime + withdrawTimeLimit) return 0;
    return req.shares;
}
```

However, neither function checks whether the vault is paused. In contrast, state-changing functions such as `deposit()` and `redeem()` are expected to revert when the contract is paused.

According to the ERC-4626 standard:

- `maxDeposit` must return 0 when deposits are disabled
- `maxRedeem` must return 0 when redemptions are disabled

This creates a mismatch where:

- View functions indicate that actions are allowed.
- Actual transactions revert due to the paused state.

Example Scenario

If the vault is paused:

```
pause();
```

Then:

- `maxDeposit(user) > 0`
- `deposit()` → reverts

Similarly:

- `maxRedeem(user) > 0`
- `redeem()` → reverts

This inconsistency can mislead integrators and users into attempting operations that will ultimately fail.



Recommendations

To align with ERC-4626 expectations and ensure consistent behavior, the protocol should incorporate pause-state checks into both functions.

Recommended fix:

```
function maxDeposit(address) external view returns (uint256) {
    if (paused()) return 0;

    uint256 currentBalance = savingsToken.balanceOf(address(this));
    if (currentBalance >= depositCap) return 0;
    return depositCap - currentBalance;
}
```

```
function maxRedeem(address owner) public view returns (uint256) {
    if (paused()) return 0;

    RedeemRequest storage req = redeemRequests[owner];
    uint256 exitTime = req.exitTime;
    if (exitTime == 0 || block.timestamp < exitTime) return 0;
    if (block.timestamp > exitTime + withdrawTimeLimit) return 0;
    return req.shares;
}
```

This ensures that:

- View functions accurately reflect the vault's operational state.
- Integrators and frontends can rely on `max*` functions for correct behavior.
- The implementation remains compliant with ERC-4626 standards.

[L-04] Incorrect asset preview in deepRebalance may prevent expected rebalancing

Description

The `deepRebalance()` function determines whether the vault holds enough pmUSD to trigger `_rebalance()` by estimating the value of module shares using `convertToAssets()`.

```
uint256 redeemableValue = baseAssetModule.convertToAssets(moduleSharesAmount);
```

This estimated value is then used in a threshold check:

```
if (preRedeemPmUSD + redeemableValue < rebalanceThreshold)
    revert BelowRebalanceThreshold();
```

However, this approach introduces a subtle inconsistency. The `convertToAssets()` function provides only a **mathematical estimate** of the asset value and does not guarantee the actual amount returned during redemption. In practice, the real redemption is performed via:



```
uint256 pmUSDReceived =  
    baseAssetModule.redeem(moduleSharesAmount, address(this), address(this));
```

Due to factors such as:

- rounding behavior
- withdrawal fees
- exchange rate updates
- external protocol mechanics

previewRedeem

Allows an on-chain **or** off-chain user to simulate the effects of their redemption at the current block, given current on-chain conditions.

MUST return as close to and no more than the exact amount of assets that would be withdrawn in a redeem call in the same transaction. I.e. redeem should **return** the same **or** more assets **as** previewRedeem **if** called **in** the same transaction.

MUST NOT account **for** redemption limits **like** those **returned** from maxRedeem **and** should always act **as** though the redemption would be accepted, regardless **if** the user has enough shares, etc.

MUST be inclusive of withdrawal fees. Integrators should be aware of the existence of withdrawal fees.

MUST NOT revert due to vault specific user/global limits. **MAY** revert due to other conditions that would also cause redeem to revert.

Note that any unfavorable discrepancy between convertToAssets **and** previewRedeem **SHOULD** be considered slippage **in** share price **or** some other **type** of condition, meaning the depositor will lose assets by redeeming.

the actual redeemed amount may be lower than the estimated value:

```
convertToAssets(shares) ≥ redeem(shares)
```

When the threshold check passes but the actual proceeds fall short, the operator or governance caller pays the full gas cost for a round-trip module redemption that converts nothing — the pmUSD is simply re-deposited back into the module unchanged.

Why This Breaks Rebalancing

The `_rebalance()` function only executes if the vault's actual pmUSD balance meets the required threshold:

```
function _rebalance() internal {  
    uint256 baseBalance = baseAsset.balanceOf(address(this));  
    uint256 pending = pendingRewards; // Use storage value (accrue already called)  
    uint256 pmUSDBalance = baseBalance > pending ? baseBalance - pending : 0;  
  
    // Step 1-3: Rebalancer  
    if (pmUSDBalance >= rebalanceThreshold && address(rebalancer) != address(0)) {
```




Rebalance uses balanceOf BaseAsset. Because the threshold check in `deepRebalance()` relies on an **overestimated value**, the following inconsistency can occur:

1. The threshold check passes using `convertToAssets()`.
2. Shares are redeemed.
3. The actual pmUSD received is lower than expected.
4. The vault balance falls below the threshold.
5. `_rebalance()` does not execute, thus lowering total assets because of the above constraints.

As a result, the function completes without performing the intended rebalance.

Example Scenario

- `rebalanceThreshold = 1000 pmUSD`
- `preRedeemPmUSD = 100`
- `moduleSharesAmount = 900`

Estimated value:

- `convertToAssets(900) = 900`
- Threshold check: $100 + 900 \geq 1000 \rightarrow$ passes

Actual redemption:

- `redeem(900) = 898`

Final balance:

- `pmUSDBalance = 998`

Since:

- $998 < 1000$

The rebalance condition fails, and `_rebalance()` is skipped despite the earlier check passing.

Recommendations

To ensure accurate and conservative estimation of redeemable assets, the protocol should replace `convertToAssets()` with `previewRedeem()` when performing threshold checks.



[L-05] Setting `withdrawTimeLimit=0` does not guarantee expiring all pending request

The `PSMVault::setWithdrawTimers()` function accepts a `withdrawTimeLimit` of zero with no minimum validation:

```
function setWithdrawTimers(uint256 _withdrawTime, uint256 _withdrawTimeLimit) external
onlyRole(GOVERNANCE_ROLE) {
    if (_withdrawTime > MAX_WITHDRAW_DELAY) revert WithdrawDelayTooLong();

    withdrawTime = _withdrawTime;
    withdrawTimeLimit = _withdrawTimeLimit;    <<@

    emit WithdrawTimersUpdated(_withdrawTime, _withdrawTimeLimit);
}
```

The specs confirm the behavior where setting `withdrawTimeLimit=0` intends to expire all pending requests:

What we borrow: The withdrawal cooldown design – storing the `exit` timestamp (not an expiration), checking against a governance-controlled withdrawal window at claim time, one request per user, and the two governance levers (`withdrawTime = 0` for immediate claims, `withdrawTimeLimit = 0` to expire all pending requests). Also the rounding pattern for round-up conversions.

The `_checkWithdrawReady()` function reverts if the current timestamp is smaller than `exitTime` or greater than `exitTime + withdrawTimeLimit`:

```
function _checkWithdrawReady(address account) internal view {
    uint256 exitTime = redeemRequests[account].exitTime;
    if (exitTime == 0 || block.timestamp < exitTime) revert WithdrawNotReady();
    if (block.timestamp > exitTime + withdrawTimeLimit) revert WithdrawWindowExpired();
}
```

However, the intention of setting `withdrawTimeLimit` to zero can be bypassed by ensuring that the withdraw transaction takes place precisely at the `exitTime` itself, which satisfies both checks, allowing the withdrawal to succeed despite the intended expiration.

Recommendations

It is recommended to only allow withdrawals when the timestamp is strictly greater than `exitTime`:

```
function _checkWithdrawReady(address account) internal view {
    uint256 exitTime = redeemRequests[account].exitTime;
    if (exitTime == 0 || block.timestamp < exitTime) revert WithdrawNotReady();

    - if (block.timestamp > exitTime + withdrawTimeLimit) revert WithdrawWindowExpired();
    + if (block.timestamp >= exitTime + withdrawTimeLimit) revert WithdrawWindowExpired();
}
```



[L-06] Rounding to zero in share conversion causes silent loss of user withdrawal value

Description

The withdrawal flow contains a critical precision issue when converting base asset amounts (pmUSD) into module shares using `convertToShares`.

In `_sendBaseAsset`, the protocol determines how many module shares correspond to a user's withdrawal entitlement:

```
uint256 sharesToSend = baseAssetModule.convertToShares(pmUSDAmount);
```

This conversion uses **floor rounding**, which creates a problematic edge case when the share price becomes significantly large (e.g., 1 share $\approx$ 1e5 pmUSD or more). In such scenarios, small withdrawal amounts can map to **0 shares** after rounding.

```
/// @dev Send base asset to receiver. If module is active, sends module shares first; any
///      portion the module cannot cover (raw pmUSD in vault) is sent as raw pmUSD.
/// @return moduleTokenSent Module shares transferred (0 if no module or module empty).
/// @return baseAssetSent Raw pmUSD transferred (0 if module covered everything).
function _sendBaseAsset(
    address receiver,
    uint256 pmUSDAmount
) internal returns (uint256 moduleTokenSent, uint256 baseAssetSent) {
    if (address(baseAssetModule) != address(0)) {
        uint256 moduleSharesBal =
IERC20(address(baseAssetModule)).balanceOf(address(this));

        if (moduleSharesBal > 0) {
            // How many module shares cover the full pmUSD entitlement – Floor (user gets
fewer)
            uint256 sharesToSend = baseAssetModule.convertToShares(pmUSDAmount); // bug
if this returns 0 loss for user // favours protocol but

            if (sharesToSend <= moduleSharesBal) {
                // Module covers full entitlement
                moduleTokenSent = sharesToSend; // 0 shares and we get nothing. bug
            } else {
                // Module can't cover – send all available module shares + raw pmUSD
remainder.
                // coveredValue uses Floor (vault-favorable: attributes less to module,
so remainder
                // is at most 1 wei larger than the "true" remainder – acceptable in
degraded state).
                moduleTokenSent = moduleSharesBal;
                uint256 coveredValue = baseAssetModule.convertToAssets(moduleSharesBal);
                baseAssetSent = pmUSDAmount > coveredValue ? pmUSDAmount -
coveredValue : 0;
            }
        } else {
            // No module shares available – all raw pmUSD
            baseAssetSent = pmUSDAmount;
        }
    }
}
```



```
if (moduleTokenSent > 0) {  
    IERC20(address(baseAssetModule)).safeTransfer(receiver, moduleTokenSent);  
}  
if (baseAssetSent > 0) {  
    baseAsset.safeTransfer(receiver, baseAssetSent);  
}
```

As a result:

- `sharesToSend` becomes `0`
- The system assumes the module can fully cover the withdrawal.
- No fallback to raw base asset transfer is triggered.
- The user receives **none of this asset**, despite having a valid entitlement.

This leads to a **silent loss of funds**, where user value is effectively trapped in the protocol.

This issue does not require malicious action—it arises naturally from:

- High share price environments in the base module
- Small or fragmented withdrawals
- Normal protocol operation over time

Recommendations

- **Handle zero-share edge case explicitly**

If `convertToShares(pmUSDAmount)` returns `0` while `pmUSDAmount` is greater than `0`, fall back to transferring raw base assets instead of assuming full coverage by the module.

Concretely, add an explicit zero-share guard at the top of the decision tree in

`_sendBaseAsset`:

```
if (sharesToSend == 0) {  
    baseAssetSent = pmUSDAmount;  
} else if (sharesToSend <= moduleSharesBal) {  
    moduleTokenSent = sharesToSend;  
} else {  
    moduleTokenSent = moduleSharesBal;  
    uint256 coveredValue = baseAssetModule.convertToAssets(moduleSharesBal);  
    baseAssetSent = pmUSDAmount > coveredValue ? pmUSDAmount - coveredValue : 0;  
}
```